

FATEC Ourinhos

Análise e Desenvolvimento de Sistemas — 3.º Semestre

SISTEMAS OPERACIONAIS

Sistemas Operacionais — Revisao Completa P1

Material de Estudo com Simulado

1. Fundamentos de Sistemas Operacionais

O que e um Sistema Operacional?

Um Sistema Operacional (SO) e o software que atua como intermediario entre o hardware e as aplicacoes do usuario. Ele cumpre dois papeis fundamentais: maquina estendida (abstrai o hardware, oferecendo interfaces simples) e gerenciador de recursos (aloca CPU, memoria e dispositivos de forma eficiente e segura entre os processos concorrentes).

Modos de Operacao: Kernel vs. Usuario

O hardware moderno opera em (pelo menos) dois niveis de privilegio. No modo kernel (supervisor), o codigo tem acesso irrestrito ao hardware — instrucoes privilegiadas, portas de I/O, registradores de controle. No modo usuario, o codigo so executa instrucoes seguras; qualquer acesso a recurso protegido gera uma excecao (trap) que transfere o controle para o kernel. Essa separacao e a base de toda a seguranca do SO.

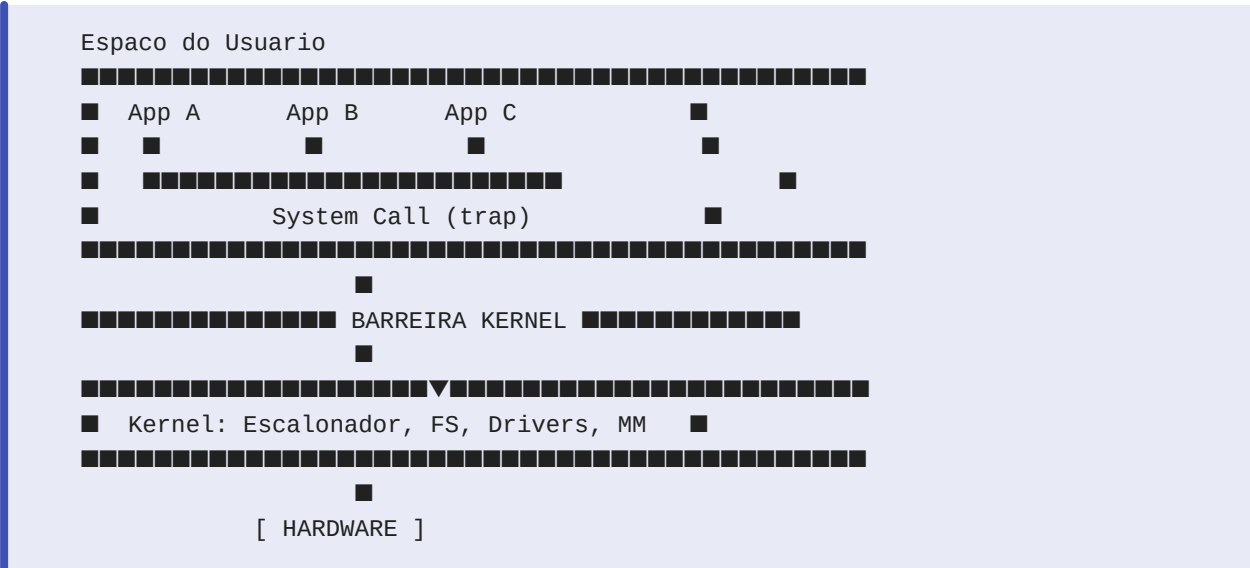


Figura 1 — Separacao espaco usuario / kernel via system calls

System Calls (Chamadas de Sistema)

System calls sao o mecanismo pelo qual um programa em modo usuario solicita servicos ao kernel. O processo executa uma instrucao especial (ex.: syscall no x86-64) que gera uma trap, salva o contexto atual, transfere a execucao para o handler no kernel e, ao terminar, restaura o contexto e retorna ao modo usuario. A tabela de syscalls do Linux tem mais de 300 entradas (read, write, fork, mmap, etc.).

Syscall	Categoria	Descricao resumida
fork()	Processos	Cria processo filho (copia do pai)

exec()	Processos	Substitui imagem do processo
read() / write()	I/O	Le/escreve em file descriptor
mmap()	Memoria	Mapeia arquivo ou memoria anonima
wait()	Processos	Pai aguarda filho terminar
kill()	Sinais	Envia sinal a um processo

Tipos e Estruturas de SO

- Monolitico: todo o kernel num unico espaco de enderecamento (Linux, BSD). Alto desempenho, menor isolamento.
- Microkernel: apenas servicos essenciais no kernel (IPC, memoria basica, escalonamento). Servicos em userspace. Mais seguro, porem com overhead de IPC (ex.: Minix, QNX, seL4).
- Hibrido: combinacao dos dois — drivers carregaveis em kernel, mas modular (Windows NT, macOS XNU).
- Exokernel: abstracao minima — aplicacoes gerenciam recursos diretamente. Pesquisa academica.

Dica: Na FATEC, a comparacao monolitico vs. microkernel e questao classica. Lembre: monolitico e mais rapido (sem IPC entre componentes), microkernel e mais seguro e modular. O Linux e monolitico, mas com suporte a modulos carregaveis (lkm).

2. Virtualizacao e Emulacao

Virtualizacao e a tecnica de criar versoes isoladas de recursos de hardware (CPU, memoria, disco) para que multiplos sistemas operacionais convidados (guests) executem sobre um mesmo hardware fisico. O software responsavel pelo gerenciamento e o hipervisor (VMM — Virtual Machine Monitor).

Hipervisor Tipo 1 (Bare-Metal)

Roda diretamente sobre o hardware, sem SO hospedeiro. Tem acesso direto a todos os recursos fisicos, o que resulta em menor overhead e maior desempenho. E o padrao em servidores de producao e datacenters.

- Exemplos: VMware ESXi, Microsoft Hyper-V (modo servidor), Xen, KVM (integrado ao kernel Linux).
- Uso tipico: nuvens publicas (AWS EC2 usa Xen/KVM; Azure usa Hyper-V).

Hipervisor Tipo 2 (Hosted)

Roda como processo em cima de um SO hospedeiro convencional. O host gerencia o hardware; o hipervisor faz chamadas ao SO hospedeiro para acessar recursos. Mais overhead, porem mais facil de instalar e usar em desktops.

- Exemplos: Oracle VirtualBox, VMware Workstation/Fusion, QEMU (modo usuario).
- Uso tipico: desenvolvimento, testes, laboratorios (como o da FATEC).

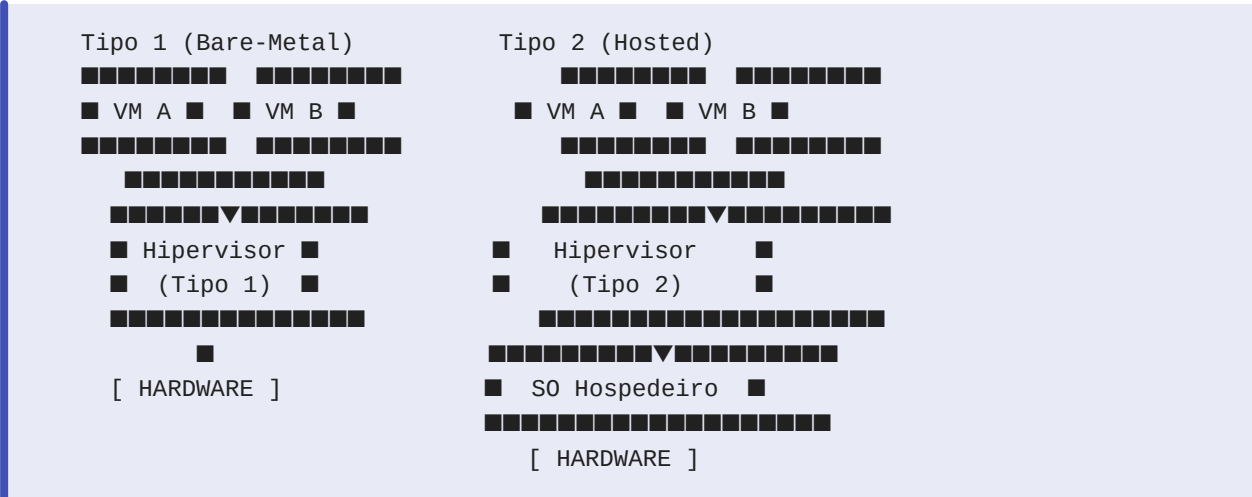


Figura 2 — Hipervisor Tipo 1 vs. Tipo 2

Containers vs. VMs

Aspecto	Virtual Machine	Container (Docker)
Isolamento	Kernel proprio por VM	Compartilha kernel do host
Overhead	Alto (GB por VM)	Baixo (MB por container)
Boot	Minutos	Segundos / milissegundos
Seguranca	Isolamento forte	Isolamento mais fraco (namespace)
Exemplo	VirtualBox, VMware	Docker, Podman, LXC

Dica: Containers usam namespaces e cgroups do Linux para isolar processos — nao sao VMs. Eles compartilham o kernel do host, portanto um exploit no kernel afeta todos os containers. VMs tem kernels separados, logo o isolamento e muito mais forte.

3. Processos e Escalonamento

O que e um Processo

Processo e um programa em execucao. O SO mantem para cada processo um PCB (Process Control Block) — estrutura que armazena tudo que o kernel precisa saber para gerencia-lo. Um processo possui quatro componentes essenciais:

- Conjunto de instrucoes: o codigo executavel carregado em memoria.
- Espaco de enderecamento: intervalo de enderecos de memoria disponiveis ao processo (stack, heap, data, text).
- Contexto de hardware: registradores, Program Counter (PC), Stack Pointer (SP) — o estado da CPU.
- Contexto de software: variaveis de ambiente, descritores de arquivo abertos, tabela de sinais.

Estados de um Processo



Figura 3 — Diagrama de estados de um processo

New: processo criado (fork). Ready: na fila, aguardando CPU. Running: executando na CPU. Waiting (Blocked): aguardando evento externo (I/O, lock). Terminated: finalizou ou foi morto (exit, SIGKILL).

Algoritmos de Escalonamento

Algoritmo	Tipo	Característica principal	Problema
FCFS	Nao-preemptivo	Primeiro a chegar, primeiro a executar	Convex effect — processos curtos atrasam
SJF	Nao-preemptivo	Menor burst de CPU executado primeiro	Starvation de processos longos; exige m
Round-Robin (RR)	Preemptivo	Quantum de tempo — fila circular	Overhead de contexto frequente se quantu
Priority	Ambos	Processo com maior prioridade executado	Solução de baixa prioridade (solucao
MLFQ	Preemptivo	Múltiplas filas com prioridades diferentes	Complexidade de implementar

Round-Robin em Detalhe

Round-Robin é o algoritmo mais usado em SOs de propósito geral (Linux usa uma variante). Cada processo recebe um quantum de tempo (tipicamente 10–100 ms). Ao esgotar o quantum, é preemptado e vai ao fim da fila de prontos. Quantum pequeno: alta responsividade, alto overhead de troca de contexto. Quantum grande: aproxima-se do FCFS.

Starvation (Inanicao)

Starvation ocorre quando um processo nunca consegue acesso ao processador porque processos de maior prioridade chegam continuamente antes dele. A solucao classica é o aging: a prioridade do processo aumenta gradualmente a medida que ele espera, garantindo que eventualmente será escalado.

4. Threads e Modelos de Concorrencia

Processo vs. Thread

Aspecto	Processo	Thread
Espaco de enderecamento	Proprio (isolado)	Compartilhado com outras threads do processo
Recursos (arquivos, sockets)	Proprios	Compartilhados no mesmo processo
Troca de contexto	Cara (muda MMU, TLB flush)	Barata (mesma MMU)
Criacao	Cara (fork duplica tudo)	Barata (pthread_create)
Falha de segmentacao	Mata so o processo	Pode matar todo o processo pai
Comunicacao	IPC (pipe, socket, shm)	Direta via memoria compartilhada

Threads compartilham o espaco de enderecamento, pilha de chamadas propria, registradores e thread-local storage. O restante (heap, descritores de arquivo, variaveis globais) é compartilhado — o que torna a sincronizacao critica.

Modelos de Thread

- N:1 (Many-to-One): N threads de usuario mapeadas em 1 thread de kernel. Implementado inteiramente em userspace (ex.: Green Threads Java legado, GNU Portable Threads). Vantagem: troca de contexto rapida. Problema grave: uma chamada de sistema bloqueante (ex.: read()) bloqueia TODAS as threads.
- 1:1 (One-to-One): cada thread de usuario tem uma thread de kernel correspondente. Paralelismo real em multicore. É o modelo do Linux (pthreads via clone()) e Windows. Desvantagem: overhead maior de criacao.
- M:N (Many-to-Many / Hibrido): M threads de usuario mapeadas em N threads de kernel (N <= M). Paralelismo real sem overhead excessivo. Complexo de implementar; usado no

Solaris e Go runtime (goroutines).

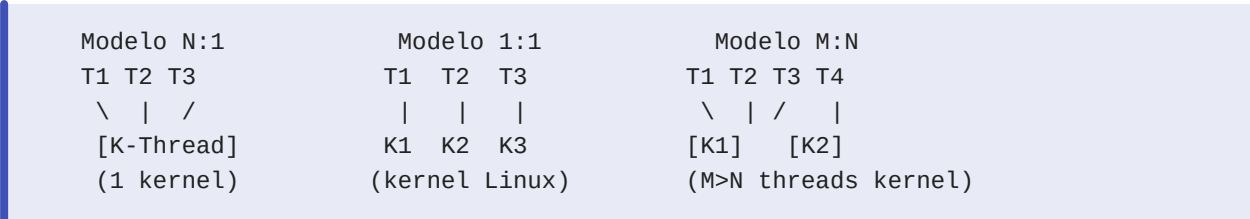


Figura 4 — Tres modelos de mapeamento thread usuario/kernel

Funcao pthread_join()

pthread_join() e a funcao POSIX que faz o thread chamador bloquear ate que o thread alvo termine. Alem de sincronizacao, ela tem papel importante no gerenciamento de recursos: libera a pilha e o bloco de controle da thread finalizaada. Sem join (ou pthread_detach()), o sistema mantem os recursos da thread mesmo apos o termino — vazamento de recursos.

```
#include <pthread.h>
#include <stdio.h>

void *tarefa(void *arg) {
    printf("Thread executando\n");
    return NULL;
}

int main(void) {
    pthread_t tid;
    pthread_create(&tid, NULL, tarefa, NULL);
    pthread_join(tid, NULL); // bloqueia ate a thread terminar
    printf("Thread concluida\n");
    return 0;
}
```

Codigo 1 — Uso basico de pthread_create + pthread_join

5. Sincronizacao: Condicoes de Corrida e Exclusao Mutua

Condicao de Corrida (Race Condition)

Uma condicao de corrida ocorre quando dois ou mais processos/threads acessam e manipulam um dado compartilhado concorrentemente e o resultado final depende da ordem especifica em que as instrucoes sao intercaladas. O problema e nao-deterministico e dificil de reproduzir — pode funcionar 99% das vezes e falhar na producao.

```
// Race condition classica: contador compartilhado
int contador = 0;

// Thread A e Thread B executam isso simultaneamente:
// 1. LOAD  contador -> reg      (le 0)
// 2. ADD   reg, 1              (calcula 1)
// 3. STORE reg -> contador      (escreve 1)

// Se as threads intercalarem entre LOAD e STORE,
// ambas leem 0, ambas calculam 1, resultado = 1 (esperado: 2)
```

Codigo 2 — Atomicidade quebrada gera race condition

Regiao Critica e os 4 Requisitos

A regioao critica e o trecho de codigo que acessa recursos compartilhados. Uma solucao correta deve satisfazer quatro propriedades: (1) Exclusao mutua — apenas um processo na regioao critica por vez; (2) Progresso — se nenhum processo esta na regioao critica, a decisao de quem entra nao pode ser adiada indefinidamente; (3) Espera limitada — um processo nao pode esperar indefinidamente para entrar; (4) Independencia de velocidade — sem suposicoes sobre a velocidade relativa dos processos.

Mutex (Mutual Exclusion Lock)

O mutex e o mecanismo de sincronizacao mais basico. Tem dois estados: travado (locked) e destravado (unlocked). Apenas o thread que adquiriu o mutex pode libera-lo (tem 'dono'). No Linux: pthread_mutex_lock() bloqueia se o mutex ja estiver travado; pthread_mutex_unlock() acorda um thread bloqueado.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int contador = 0;

void *incrementar(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);    // entra na regioao critica
        contador++;                  // operacao atomica agora
        pthread_mutex_unlock(&lock);  // sai da regioao critica
    }
    return NULL;
}
```

Codigo 3 — Protecao de regioao critica com pthread_mutex

Semaforo vs. Mutex

Aspecto	Mutex	Semaforo
Natureza	Lock binario (0 ou 1)	Contador inteiro (>= 0)

Dono	Sim — so quem travou pode destravar	Não — qualquer thread pode fazer signal()
Uso principal	Exclusao mutua (1 thread por vez)	Controle de recursos (N tokens disponiveis)
Operacoes	lock() / unlock()	wait()/P() decrementa; signal()/V() incrementa
Exemplo tipico	Proteger secao critica	Limitar conexoes simultaneas ao banco (pool)

Spinlock

Spinlock e uma forma de busy waiting: o thread fica em loop testando o lock ate ele ser liberado. Vantagem: sem overhead de troca de contexto para esperas muito curtas. Desvantagem: desperdicao de CPU se a espera for longa. Usado em kernels e drivers onde a regio critica e minuscula e o contexto de interrupcao nao permite dormir.

Dica: Resumo rapido: use mutex para exclusao mutua simples (1 thread por vez). Use semaforo binario quando nao importa quem libera. Use semaforo contador para controlar acesso a N recursos. Nunca use spinlock em userspace — so em kernel com interrupcoes desabilitadas.

6. Problemas Classicos de IPC

Jantar dos Filósofos

5 filosofos sentam em mesa circular. Entre cada par ha 1 garfo (5 garfos no total). Para comer, um filosofo precisa dos 2 garfos adjacentes. O problema modela a alocação de recursos compartilhados entre processos concorrentes e o surgimento de deadlock.

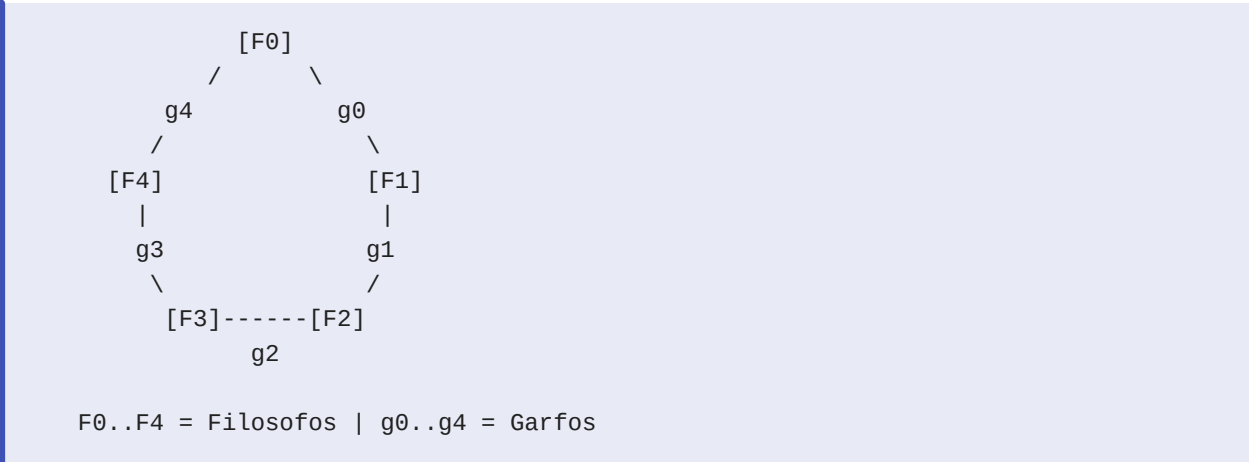


Figura 5 — Jantar dos Filósofos: 5 filósofos, 5 garfos

Como o Deadlock surge

Se todos os 5 filósofos pegarem o garfo da esquerda simultaneamente, cada um terá 1 garfo e ficará esperando o garfo da direita — que está com o vizinho. Forma-se uma espera circular: F0 espera g0 (com F1), F1 espera g1 (com F2), ..., F4 espera g4 (com F0). Nenhum pode avançar: deadlock completo.

Soluções ao Jantar dos Filósofos

- Ordenação de recursos: numerar os garfos 0..4. Filósofos sempre pegam o de menor número primeiro. Quebra a espera circular (o filósofo entre g4 e g0 pegaria g0 antes de g4 — sentido diferente dos outros).
- Filósofo assimétrico: um filósofo pega garfo direito primeiro, os demais pegam esquerdo. Elimina o ciclo.
- Semaforo de capacidade: apenas 4 filósofos podem sentar simultaneamente (semaforo inicializado com 4). Garante que ao menos 1 pode comer.
- Monitor com variável de condição: filósofo só tenta pegar garfos se ambos estiverem disponíveis (verificação atômica dentro do monitor).

Problema dos Leitores e Escritores

Múltiplos leitores podem acessar o recurso simultaneamente (leitura não gera inconsistência). Escritores precisam de acesso exclusivo. O problema é balancear a prioridade: se leitores têm prioridade máxima, escritores sofrem starvation (novos leitores continuam chegando). Solução: quando um escritor sinaliza intenção, leitores subsequentes devem esperar — garantindo que o escritor acesse o recurso em tempo finito.

Barbeiro Adormecido (Sleeping Barber)

Barbearia com 1 barbeiro e N cadeiras de espera. Se não há clientes, o barbeiro dorme. Quando um cliente chega: se há cadeira livre, ele senta e acorda o barbeiro; caso contrário, vai embora. Modela o padrão produtor-consumidor com buffer limitado. O problema central é a sincronização do acordar/dormir sem race condition.

7. Deadlocks (Impasses)

Um deadlock é uma situação em que um conjunto de processos fica bloqueado permanentemente: cada processo aguarda um recurso que está retido por outro processo do conjunto, formando uma dependência circular sem saída.

As 4 Condições de Coffman (Todas Necessárias)

- 1. Exclusão Mútua: o recurso só pode ser usado por um processo de cada vez (não é compartilhável).

- 2. Segurar e Esperar (Hold & Wait): um processo que ja possui pelo menos um recurso fica aguardando a liberacao de outro recurso retido por terceiro.
- 3. Nao Preempcao (No Preemption): recursos nao podem ser tomados forcosamente de um processo — so sao liberados voluntariamente.
- 4. Espera Circular (Circular Wait): existe uma cadeia fechada de processos $P1 \rightarrow P2 \rightarrow \dots \rightarrow Pn \rightarrow P1$, onde cada Pi aguarda recurso retido por $P(i+1)$.

Dica: Para deadlock ocorrer, as QUATRO condicoes devem ser satisfeitas simultaneamente. Basta negar UMA delas para prevenir deadlock. Esta e a base da estrategia de prevencao.

Grafo de Alocacao de Recursos – deadlock:

```

P1 ---solicita---> R1 ---alocado---> P2
  ^                               |
  |                               | solicita
alocado                         |
  |                               v
R2 <---solicita--- P3 <---alocado--- R2

Ciclo P1->R1->P2->R2->P3->R1: deadlock!

```

Figura 6 — Grafo de alocao com ciclo (deadlock)

4 Estrategias para Lidar com Deadlocks

1) Prevencao de Deadlocks

Estruturar o sistema para tornar impossivel a ocorrencia de pelo menos uma das quatro condicoes. Exemplos: exigir que um processo declare todos os recursos que precisara de uma vez antes de executar (elimina Hold & Wait); impor uma ordem total sobre os recursos e exigir que sejam requisitados em ordem crescente (elimina Espera Circular).

2) Evasao de Deadlocks (Esquiva)

O sistema analisa dinamicamente cada requisicao de recurso e so a concede se o estado resultante for 'seguro' (safe state). Um estado e seguro se existe pelo menos uma sequencia de execucao que permite que todos os processos terminem. O Algoritmo do Banqueiro de Dijkstra e o exemplo canonico: o SO age como um banqueiro que so empresta recursos se ainda puder atender todos os clientes.

3) Deteccao e Recuperacao

Permite que deadlocks ocorram, mas o sistema monitora periodicamente o estado dos recursos (construindo o grafo de alocao e detectando ciclos). Ao detectar um deadlock: (a) aborta um ou mais processos para quebrar o ciclo, ou (b) faz preempcao de recursos —

reverte um processo a um checkpoint anterior (rollback).

4) Política do Avestruz (Ignorar o Problema)

Abordagem pragmática: assume-se que deadlocks são suficientemente raros para que o custo de prevenir/detectar não valha. O sistema simplesmente não faz nada — cabe ao usuário reiniciar o processo ou o sistema. Adotada por Unix e Windows para a maioria dos recursos de usuário, pois deadlocks reais são raros na prática.

Estrategia	Overhead	Restricao ao programador	Uso tipico
Prevencao	Baixo em runtime	Alta (regras rigidas)	Sistemas de tempo real
Evasao (Banker)	Medio (analise por requisicao)	Medio (declarar maximos)	Academico / teorico
Deteccao + Recuperacao	Periodico (exec. do detector)	Baixa	Bancos de dados, DBMSs
Avestruz	Nenhum	Nenhuma	Unix, Windows, uso geral

SIMULADO

Questão 1 (Dissertativa)

Explique a diferença entre modo kernel e modo usuário em um sistema operacional moderno. Por que essa separação é fundamental para a segurança e estabilidade do sistema? Descreva o mecanismo de transição entre os dois modos.

Questão 2 (Múltipla Escolha)

Sobre os modelos de thread, assinale a alternativa CORRETA: a) No modelo N:1, um bloqueio de I/O em uma thread não afeta as demais threads do processo. b) No modelo 1:1, cada thread de usuário tem uma thread de kernel correspondente, permitindo paralelismo real em sistemas multicore. c) O modelo M:N é o mais simples de implementar pois não exige comunicação com o kernel. d) Threads de usuário são sempre mais rápidas que threads de kernel em qualquer cenário.

Questão 3 (Dissertativa)

O que é uma condição de corrida (race condition)? Apresente um exemplo concreto e explique como o uso de mutex resolve o problema. Qual a diferença entre mutex e semaforo?

Questão 4 (Dissertativa)

No problema do Jantar dos Filósofos com 5 filósofos, descreva como surge o deadlock e apresente duas soluções distintas, explicando por que cada uma funciona.

Questão 5 (Dissertativa)

Quais são as quatro condições necessárias para a ocorrência de deadlock (condições de Coffman)? Para cada uma, descreva como sua negação pode prevenir o deadlock.

Questão 6 (Múltipla Escolha)

Sobre algoritmos de escalonamento, qual afirmação é INCORRETA? a) FCFS pode causar o 'convoy effect', onde processos curtos esperam muito atrás de processos longos. b) Round-Robin com quantum muito grande se comporta de forma similar ao FCFS. c) SJF garante starvation zero pois sempre prioriza os menores jobs. d) Aging é uma técnica usada para evitar starvation em escalonamento por prioridade.

Questão 7 (Dissertativa)

Compare hipervisores do Tipo 1 e Tipo 2, apresentando exemplos de cada um. Em qual cenário cada tipo é mais adequado e por que? Qual a principal diferença entre uma VM e um container (Docker)?

Questão 8 (Múltipla Escolha)

Sobre deadlocks, qual das estratégias abaixo NENHUM sistema operacional de uso geral (como Linux ou Windows) adota como abordagem principal? a) Política do Avestruz — ignorar o problema. b) Detecção e recuperação via aborto de processos. c) Prevenção estrutural negando a condição de espera circular em todos os recursos do sistema. d) Evasão via Algoritmo do Banqueiro aplicado a recursos de usuário.

Questão 9 (Dissertativa)

Descreva os cinco estados de um processo e as transições entre eles. Em quais transições o escalonador (scheduler) está envolvido diretamente? O que é starvation e como o aging a resolve?

Questão 10 (Dissertativa)

Explique o papel da funcao `pthread_join()`. O que acontece com os recursos de uma thread que termina sem que `join()` ou `detach()` seja chamado? Qual a diferenca entre thread joinable e thread detached?

GABARITO COMENTADO

Questão 1 (Dissertativa):

Em modo usuario o processo executa instrucoes nao privilegiadas e nao pode acessar diretamente hardware ou memoria do kernel. Em modo kernel o codigo tem acesso irrestrito a todos os recursos. A separacao impede que um programa mal-comportado (ou malicioso) corrompa o kernel ou outros processos. A transicao ocorre via trap (system call): o processo executa instrucao especial (syscall/int 0x80), o hardware salva o contexto, aumenta o nivel de privilegio e desvia para o handler no kernel. Ao terminar, o kernel restaura o contexto e retorna ao modo usuario via iret/sysret.

Questão 2 (Múltipla Escolha):

Alternativa B. No modelo 1:1 (usado pelo Linux com pthreads), cada thread de usuario mapeia para uma thread de kernel, permitindo execucao paralela real em multiplos nucleos. A) e falsa: no N:1, uma syscall bloqueante congela TODAS as threads. C) e falsa: M:N e o mais complexo. D) e falsa: threads de usuario sao mais rapidas para troca de contexto mas bloqueiam todo o processo em syscalls bloqueantes.

Questão 3 (Dissertativa):

Race condition: dois ou mais threads acessam dado compartilhado concorrentemente e o resultado depende da ordem de intercalacao das instrucoes. Exemplo: dois threads incrementando um contador — ambos leem o valor antes de qualquer um escrever, resultado final e 1 em vez de 2. O mutex protege a regio critica: apenas um thread executa o bloco lock/unlock por vez, tornando a operacao atomica. Diferenca: mutex tem 'dono' (so quem travou pode destravar) e e binario; semaforo e um contador inteiro sem dono, usado para controlar acesso a N recursos.

Questão 4 (Dissertativa):

Deadlock surge quando todos os 5 filosofos pegam o garfo esquerdo simultaneamente: cada um tem 1 garfo e espera o da direita (retido pelo vizinho) — espera circular. Solucao 1: ordenacao de recursos — numerar garfos 0..4 e exigir que o de menor indice seja pego primeiro. Um filosofo (entre g4 e g0) pegaria g0 antes de g4, sentido contrario, quebrando a circularidade. Solucao 2: semaforo de capacidade inicializado com 4 — no maximo 4 filosofos sentam, garantindo que pelo menos 1

pode comer (tem ambos os garfos disponiveis).

Questão 5 (Dissertativa):

1. Exclusao Mutua: recurso usado por apenas 1 processo. Negacao: tornar recursos compartilhaveis (nem sempre possivel, ex.: impressora). 2. Hold & Wait: processo retendo recurso aguarda outro. Negacao: processo deve requisitar todos os recursos de uma vez antes de executar (ou liberar tudo antes de nova requisicao). 3. Nao Preempcao: recursos nao podem ser tomados a forza. Negacao: permitir preempcao (viavel para recursos de estado salvo, como CPU; inviavel para impressoras). 4. Espera Circular: cadeia fechada de esperas. Negacao: impor ordem total nos recursos e exigir requisicao em ordem crescente — impossibilita ciclo.

Questão 6 (Múltipla Escolha):

Alternativa C. SJF (Shortest Job First) prioriza processos de menor burst de CPU, o que pode causar starvation em processos longos — eles ficam preteridos indefinidamente se processos curtos continuam chegando. Aging (envelhecimento) e justamente a solucao para starvation em algoritmos de prioridade (alternativa D, verdadeira). A e B sao verdadeiras.

Questão 7 (Dissertativa):

Tipo 1 (bare-metal): roda diretamente no hardware sem SO hospedeiro. Exemplos: VMware ESXi, Xen, KVM. Menor overhead, maior desempenho e seguranca — adequado para servidores de producao e nuvens. Tipo 2 (hosted): roda como processo sobre SO hospedeiro. Exemplos: VirtualBox, VMware Workstation. Mais facil de instalar, adequado para desenvolvimento e laboratorios. VM vs. Container: VMs tem kernel proprio por instancia (isolamento forte, maior overhead). Containers compartilham o kernel do host via namespaces/cgroups (isolamento fraco, overhead minimo).

Questão 8 (Múltipla Escolha):

Alternativas C e D. A prevencao estrutural total e o Algoritmo do Banqueiro sao impraticaveis em SOs de uso geral: a prevencao imporia restricoes severas a todos os programas; o Banqueiro exige que processos declarem antecipadamente o maximo de recursos necessarios (impratico). Linux e Windows adotam principalmente a Politica do Avestruz (A) para recursos de usuario e deteccao limitada em subsistemas especificos (B), como o lockdep do kernel Linux.

Questão 9 (Dissertativa):

New: processo criado. Ready: apto, aguardando CPU na fila de prontos. Running: executando na CPU. Waiting/Blocked: aguardando evento externo (I/O, sinal, mutex). Terminated: finalizou. Transicoes: New->Ready (admissao), Ready->Running (dispatch pelo escalonador), Running->Ready (preempcao por quantum ou prioridade — escalonador envolvido), Running->Waiting (I/O voluntario), Waiting->Ready (I/O completo), Running->Terminated. Starvation: processo nunca recebe CPU porque processos de maior prioridade chegam continuamente. Aging: a prioridade do processo aumenta gradualmente com o tempo de espera, garantindo que eventualmente ele sera escalonado.

Questão 10 (Dissertativa):

`pthread_join()` faz o thread chamador bloquear ate que o thread alvo termine, sincronizando a execucao. Alem disso, e responsavel por liberar os recursos da thread finalizada (pilha, bloco de controle). Sem `join()` ou `pthread_detach()`, o sistema mantem esses recursos mesmo apos o termino — vazamento de recursos (thread zumbi). Thread joinable (padrao): aguarda que outro thread chame `join()` para liberar recursos; permite recuperar o valor de retorno. Thread detached: recursos sao liberados automaticamente ao terminar; nao e possivel fazer `join()` nem obter o retorno.